

UVMF YAML Structure & Format Reference Manual

Complete Spec-Driven UVMF Project: AXI4-Stream

*Practical generator structure • reusable interfaces • environment • bench • custom sequences/tests •
regression/coverage traceability*

Updated: September 24, 2026

Version basis — The YAML examples use the classic UVMF code-generator structure documented by UVMF 2019.4-era material. Exact optional keys can vary by installed UVMF/Questa release; treat `$UVMF_HOME/docs/UVMF_Code_Generator_YAML_Reference.pdf` as the final schema authority for your installation.

1. What This Update Changes

This revision converts the previous mixed AXI4-Stream/ALU material into one coherent AXI4-Stream project and separates generator-supported YAML from project-specific regression metadata.

Area	Updated rule
Interface YAML	Use UVMF interface keys such as clock, reset, reset_assertion_level, parameters, ports, transaction_vars, transaction_constraints and config_vars.
Environment YAML	Instantiate agents, predictor/analysis components, scoreboards and TLM connections. Agent protocol role is defined here.
Bench YAML	Define top_env, BFM ACTIVE/PASSIVE mode, clock and reset behavior. DUT instantiation/wiring is completed in generated hdl_top.sv for the classic flow.
Sequences / tests	Custom verification behavior lives primarily in SystemVerilog sequence/test classes generated from the bench/environment skeleton; a generic sequences: YAML block is not assumed.
Coverage	Classic UVMF generates an interface transaction coverage class; customize/extend its SystemVerilog covergroup as needed. Do not assume an arbitrary coverage: YAML block is portable.
randc	Interface YAML exposes isrand, not a portable randc key. Cyclic behavior is implemented in custom SystemVerilog or deterministic sequence logic.
Regression list	Keep regression orchestration separate from yaml2uvmf input unless using a release-specific VRM testlist schema. This manual provides a safe project-owned regression manifest plus a traceability matrix.

2. Project Definition and DUT Assumptions

Reference DUT: an AXI4-Stream pipeline/register-slice style block. It accepts beats on S_AXIS and emits the same beats, in order, on M_AXIS after implementation-defined latency. The verification architecture remains valid for a transform block if the predictor is changed accordingly.

Parameter	Reference value	Meaning
DATA_WIDTH	32	TDATA width in bits
KEEP_WIDTH	4	TDATA byte-lane qualifiers
ID_WIDTH	4	Optional TID width
DEST_WIDTH	4	Optional TDEST width
USER_WIDTH	4	Optional TUSER width
Clock	100 MHz	10 ns period
Reset	Active-low	areset_n

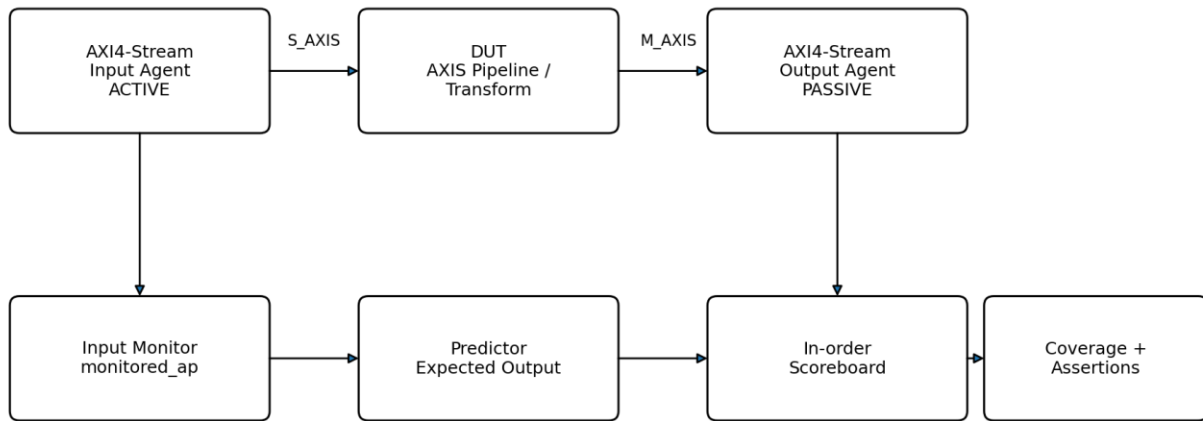
3. Verification Specification (Source of Truth)

The project is driven from explicit requirements. Each requirement maps to stimulus, checking, functional coverage, and where appropriate an assertion.

ID	Requirement	Checker	Coverage target
----	-------------	---------	-----------------

ID	Requirement	Checker	Coverage target
AXIS-REQ-001	Transfer occurs only on a rising edge where TVALID && TREADY are both 1.	Handshake monitor + scoreboard	cp_handshake
AXIS-REQ-002	While TVALID=1 and TREADY=0, the master holds payload and sideband information stable.	SVA hold-under-stall	cp_stall_len
AXIS-REQ-003	TLAST identifies the final beat of a generated packet.	Packet-aware sequence + monitor	cp_pkt_len x cp_last
AXIS-REQ-004	TKEEP identifies valid byte lanes; directed tests include full and partial final beats.	Transaction compare	cp_keep
AXIS-REQ-005	After reset assertion, source TVALID is inactive and no stale transfer is accepted.	Reset assertion + monitor flush	cp_reset
AXIS-REQ-006	DUT preserves beat order and payload for the reference pipeline DUT.	Predictor + in-order scoreboard	cp_data_class / count
AXIS-REQ-007	TID/TDEST/TUSER are preserved for accepted beats when enabled.	Predictor + scoreboard	cp_id, cp_dest, cp_user
AXIS-REQ-008	Backpressure does not drop or duplicate accepted beats.	Random ready pattern + scoreboard count	cross stall x packet length

4. Verification Architecture



Spec -> stimulus -> protocol checks -> prediction -> comparison -> coverage closure

Figure 1. Spec-driven AXI4-Stream UVMF architecture.

- axis_in_agent: ACTIVE initiator; generates source traffic and monitors accepted input transfers.
- axis_out_agent: PASSIVE monitor in the reference bench; M_AXIS TREADY is tied high in hdl_top.sv. For randomized backpressure, replace the tie-high with an active responder strategy.
- axis_predictor: converts each accepted input transaction into the expected output transaction. For a transparent pipeline this is a field-for-field copy.
- axis_sb: UVMF in-order scoreboard comparing predicted and observed output beats.

- Coverage/assertions: UVMF generates transaction coverage support for an interface; customize or extend the SystemVerilog coverage model. Protocol/property assertions are also implemented in SystemVerilog.

5. Complete Project File Structure

```

axi4stream_uvmf/
├── yaml/
│   ├── axis_in_interface.yaml
│   ├── axis_out_interface.yaml
│   ├── axis_predictor.yaml
│   ├── axis_environment.yaml
│   └── axis_bench.yaml
├── rtl/
│   └── axis_pipeline.sv
├── custom/
│   ├── axis_sequences.svh
│   ├── axis_tests.svh
│   ├── axis_coverage.svh
│   └── axis_assertions.sv
├── regression/
│   └── axis_regression_plan.yaml # project metadata; not yaml2uvmf input
└── uvmf_template_output/        # generated by yaml2uvmf.py

```

6. Interface YAML — S_AXIS Input

The input interface describes the physical protocol signals and the beat-level transaction used by the source agent. The UVMF driver/monitor BFM's translate between transaction fields and pins.

yaml/axis_in_interface.yaml

```

uvmf:
  interfaces:
    axis_in:
      clock: "aclk"
      reset: "areset_n"
      reset_assertion_level: "False"
      veloce_ready: "True"

      parameters:
        - name: "DATA_WIDTH"
          type: "int"
          value: "32"
        - name: "KEEP_WIDTH"
          type: "int"
          value: "4"
        - name: "ID_WIDTH"
          type: "int"
          value: "4"
        - name: "DEST_WIDTH"
          type: "int"
          value: "4"
        - name: "USER_WIDTH"
          type: "int"
          value: "4"

      ports:
        - {name: "tvalid", dir: "output", width: "1", reset_value: "1'b0"}
        - {name: "tready", dir: "input", width: "1", reset_value: "1'b0"}
        - {name: "tdata", dir: "output", width: "DATA_WIDTH", reset_value: "'0"}
        - {name: "tkeep", dir: "output", width: "KEEP_WIDTH", reset_value: "'0"}
        - {name: "tlast", dir: "output", width: "1", reset_value: "1'b0"}
        - {name: "tid", dir: "output", width: "ID_WIDTH", reset_value: "'0"}
        - {name: "tdest", dir: "output", width: "DEST_WIDTH", reset_value: "'0"}
        - {name: "tuser", dir: "output", width: "USER_WIDTH", reset_value: "'0"}

      transaction_vars:

```

```

- {name: "data", type: "bit [DATA_WIDTH-1:0]", isrand: "True", iscompare: "True"}
- {name: "keep", type: "bit [KEEP_WIDTH-1:0]", isrand: "True", iscompare: "True"}
- {name: "last", type: "bit", isrand: "True", iscompare: "True"}
- {name: "id", type: "bit [ID_WIDTH-1:0]", isrand: "True", iscompare: "True"}
- {name: "dest", type: "bit [DEST_WIDTH-1:0]", isrand: "True", iscompare: "True"}
- {name: "user", type: "bit [USER_WIDTH-1:0]", isrand: "True", iscompare: "True"}
- {name: "gap", type: "int unsigned", isrand: "True", iscompare: "False"}

transaction_constraints:
- name: "keep_nonzero_c"
  value: "{ keep != '0; }"
- name: "gap_range_c"
  value: "{ gap inside {[0:8]}; }"

config_vars:
- {name: "max_wait_cycles", type: "int unsigned", value: "1000"}

```

7. Interface YAML — M_AXIS Output

The output interface is monitor-oriented in the reference project. Transaction fields are not randomized because they represent observed DUT output. TREADY is sampled as an input signal; hdl_top.sv drives it high for the baseline bench.

yaml/axis_out_interface.yaml

```

uvmf:
  interfaces:
    axis_out:
      clock: "aclk"
      reset: "areset_n"
      reset_assertion_level: "False"
      veloce_ready: "True"

  parameters:
    - {name: "DATA_WIDTH", type: "int", value: "32"}
    - {name: "KEEP_WIDTH", type: "int", value: "4"}
    - {name: "ID_WIDTH", type: "int", value: "4"}
    - {name: "DEST_WIDTH", type: "int", value: "4"}
    - {name: "USER_WIDTH", type: "int", value: "4"}

  ports:
    - {name: "tvalid", dir: "input", width: "1", reset_value: "1'b0"}
    - {name: "tready", dir: "input", width: "1", reset_value: "1'b0"}
    - {name: "tdata", dir: "input", width: "DATA_WIDTH", reset_value: "'0"}
    - {name: "tkeep", dir: "input", width: "KEEP_WIDTH", reset_value: "'0"}
    - {name: "tlast", dir: "input", width: "1", reset_value: "1'b0"}
    - {name: "tid", dir: "input", width: "ID_WIDTH", reset_value: "'0"}
    - {name: "tdest", dir: "input", width: "DEST_WIDTH", reset_value: "'0"}
    - {name: "tuser", dir: "input", width: "USER_WIDTH", reset_value: "'0"}

  transaction_vars:
    - {name: "data", type: "bit [DATA_WIDTH-1:0]", isrand: "False", iscompare: "True"}
    - {name: "keep", type: "bit [KEEP_WIDTH-1:0]", isrand: "False", iscompare: "True"}
    - {name: "last", type: "bit", isrand: "False", iscompare: "True"}
    - {name: "id", type: "bit [ID_WIDTH-1:0]", isrand: "False", iscompare: "True"}
    - {name: "dest", type: "bit [DEST_WIDTH-1:0]", isrand: "False", iscompare: "True"}
    - {name: "user", type: "bit [USER_WIDTH-1:0]", isrand: "False", iscompare: "True"}

```

AXI4-Stream rule — A beat is sampled only when TVALID && TREADY is true at the active clock edge. The monitor BFM must not emit a transaction for cycles where that handshake is false.

8. Predictor Utility Component YAML

yaml/axis_predictor.yaml

```
uvmf:
  util_components:
    axis_predictor:
      type: predictor
      analysis_exports:
        - name: "axis_in_agent_ae"
          type: "axis_in_transaction #()"
      analysis_ports:
        - name: "axis_sb_ap"
          type: "axis_out_transaction #()"
```

The generated predictor write method is a customization point. For the reference pipeline DUT, copy data/keep/last/id/dest/user from the accepted input transaction to a newly created output transaction and write it on axis_sb_ap.

Generated predictor customization (illustrative)

```
virtual function void write_axis_in_agent_ae(axis_in_transaction #() t);
  axis_out_transaction #() exp;
  exp = axis_out_transaction #()::type_id::create("exp");
  exp.data = t.data;
  exp.keep = t.keep;
  exp.last = t.last;
  exp.id = t.id;
  exp.dest = t.dest;
  exp.user = t.user;
  axis_sb_ap.write(exp);
endfunction
```

9. Environment YAML

yaml/axis_environment.yaml

```
uvmf:
  environments:
    axis_env:
      agents:
        - name: "axis_in_agent"
          type: "axis_in"
          initiator_responder: "INITIATOR"
        - name: "axis_out_agent"
          type: "axis_out"
          initiator_responder: "RESPONDER"

      analysis_components:
        - name: "axis_pred"
          type: "axis_predictor"

      scoreboards:
        - name: "axis_sb"
          sb_type: "uvmf_in_order_scoreboard"
          trans_type: "axis_out_transaction #()"

      tlm_connections:
        - driver: "axis_in_agent.monitored_ap"
          receiver: "axis_pred.axis_in_agent_ae"
        - driver: "axis_pred.axis_sb_ap"
          receiver: "axis_sb.expected_analysis_export"
        - driver: "axis_out_agent.monitored_ap"
          receiver: "axis_sb.actual_analysis_export"
```

Role vs activity — initiator_responder describes protocol role in the reusable environment. ACTIVE/PASSIVE is selected in the bench for the particular simulation configuration.

10. Bench YAML

yaml/axis_bench.yaml

```
uvmf:
  benches:
    axis_pipe:
      top_env: "axis_env"

      active_passive:
        - bfm_name: "axis_in_agent"
          value: "ACTIVE"
        - bfm_name: "axis_out_agent"
          value: "PASSIVE"

      clock_half_period: 5ns
      clock_phase_offset: 0ns
      reset_assertion_level: "False"
      reset_duration: 100ns
      interface_params: []
```

The generated bench creates hdl_top.sv, hvl_top.sv, test/sequence package skeletons, file lists and simulation support. The classic UVMF flow does not require the bench YAML to contain a generic dut: port map.

11. Generate the UVMF Project

Linux / shell

```
python "$UVMF_HOME/scripts/yaml2uvmf.py" \
  yaml/axis_in_interface.yaml \
  yaml/axis_out_interface.yaml \
  yaml/axis_predictor.yaml \
  yaml/axis_environment.yaml \
  yaml/axis_bench.yaml
```

Windows CMD

```
python "%UVMF_HOME%\scripts\yaml2uvmf.py" ^
  yaml\axis_in_interface.yaml ^
  yaml\axis_out_interface.yaml ^
  yaml\axis_predictor.yaml ^
  yaml\axis_environment.yaml ^
  yaml\axis_bench.yaml
```

- Re-running the generator is intended to preserve UVMF custom-code regions; still keep the generated tree under version control or review diffs before accepting regeneration.
- Resolve generator schema errors before editing generated code. A misspelled agent, interface or TLM endpoint should be fixed in YAML, not patched downstream.

12. Complete hdl_top.sv DUT Hookup

After generation, instantiate the DUT and connect the generated signal-bundle interfaces. Exact generated instance names vary slightly by UVMF release, so use the generated hdl_top.sv as the authoritative naming template.

```
// Illustrative DUT hookup inside generated hdl_top.sv custom region
logic m_axis_tready;
assign m_axis_tready = 1'b1; // baseline: no output backpressure

axis_pipeline #(
    .DATA_WIDTH(32), .ID_WIDTH(4), .DEST_WIDTH(4), .USER_WIDTH(4)
) dut (
    .aclk           (aclk),
    .areset_n       (areset_n),

    .s_axis_tvalid   (axis_in_if.tvalid),
    .s_axis_tready   (axis_in_if.tready),
    .s_axis_tdata     (axis_in_if.tdata),
    .s_axis_tkeep     (axis_in_if.tkeep),
    .s_axis_tlast     (axis_in_if.tlast),
    .s_axis_tid       (axis_in_if.tid),
    .s_axis_tdest     (axis_in_if.tdest),
    .s_axis_tuser     (axis_in_if.tuser),

    .m_axis_tvalid   (axis_out_if.tvalid),
    .m_axis_tready   (m_axis_tready),
    .m_axis_tdata     (axis_out_if.tdata),
    .m_axis_tkeep     (axis_out_if.tkeep),
    .m_axis_tlast     (axis_out_if.tlast),
    .m_axis_tid       (axis_out_if.tid),
    .m_axis_tdest     (axis_out_if.tdest),
    .m_axis_tuser     (axis_out_if.tuser)
);
assign axis_out_if.tready = m_axis_tready;
```

13. BFM Protocol Implementation

UVMF generates BFM skeletons; the protocol-specific pin behavior must be implemented in the designated custom regions. For AXI4-Stream, the driver must hold TVALID and payload stable until a handshake occurs.

Input driver behavior (conceptual)

```
task initiate_and_get_response(input axis_in_transaction_s req,
                              output axis_in_transaction_s rsp);
    repeat (req.gap) @(posedge aclk);

    tvalid <= 1'b1;
    tdata  <= req.data;
    tkeep  <= req.keep;
    tlast  <= req.last;
    tid    <= req.id;
    tdest  <= req.dest;
    tuser  <= req.user;

    do @(posedge aclk); while (!tready);

    tvalid <= 1'b0;
    tlast  <= 1'b0;
    rsp = req;
endtask
```

Output monitor behavior (conceptual)

```
task do_monitor(output axis_out_transaction_s txn);
    forever begin
        @(posedge aclk);
        if (!areset_n) continue;
        if (tvalid && tready) begin
            txn.data = tdata;
            txn.keep = tkeep;
            txn.last = tlast;
            txn.id   = tid;
        end
    end
endtask
```

```

    txn.dest = tdest;
    txn.user = tuser;
    return;
  end
end
endtask

```

Implementation detail — Generated BFM method signatures and transaction structs are release-specific. Copy the generated function/task prototype exactly, then insert the protocol behavior inside the provided UVMF custom region.

14. Interface and Top-Level Sequences — SystemVerilog

Protocol-specific stimulus belongs in an interface sequence derived from the generated `axis_in_sequence_base`. Bench-level scenario control belongs in a top-level virtual sequence derived from the generated `axis_pipe_sequence_base`. This follows the UVMF sequence hierarchy and avoids inventing unsupported YAML sequence keys.

Smoke sequence pattern

```

class axis_in_smoke_sequence extends axis_in_sequence_base;
  `uvm_object_utils(axis_in_smoke_sequence)

  task body();
    repeat (20) begin
      axis_in_transaction #() tr;
      tr = axis_in_transaction #()::type_id::create("tr");
      start_item(tr);
      if (!tr.randomize() with {
        keep == '1;
        gap inside {[0:2]};
      }) `uvm_fatal("RAND", "axis_in_smoke_sequence randomize failed")
      finish_item(tr);
    end
  endtask
endclass

```

Packet sequence pattern

```

class axis_in_packet_sequence extends axis_in_sequence_base;
  rand int unsigned packet_len;
  constraint packet_len_c { packet_len inside {[1:64]}; }

  task body();
    assert(randomize());
    for (int beat = 0; beat < packet_len; beat++) begin
      axis_in_transaction #() tr;
      tr = axis_in_transaction #()::type_id::create($sformatf("tr_%0d", beat));
      start_item(tr);
      if (!tr.randomize() with {
        last == (beat == packet_len-1);
        if (!last) keep == '1;
        gap inside {[0:8]};
      }) `uvm_fatal("RAND", "axis_in_packet_sequence randomize failed")
      finish_item(tr);
    end
  endtask
endclass

```

Top-level bench scenario sequences should extend the generated `axis_pipe_sequence_base`. Use the generated `example_derived_test_sequence` and `axis_pipe_sequence_base.svh` as the naming/API template; UVMF provides

the active agent configuration and sequencer handles there. The top-level sequence creates and starts `axis_in_smoke_sequence` / `axis_in_packet_sequence` on the AXI4-Stream input sequencer, and can call configuration utilities such as `wait_for_reset()` and `wait_for_num_clocks()`.

Derived test selects a top-level sequence (illustrative)

```
class axis_smoke_test extends test_top;
  `uvm_component_utils(axis_smoke_test)

  function new(string name = "axis_smoke_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    // Override the generated default top-level sequence with the scenario sequence.
    axis_pipe_sequence_base::type_id::set_type_override(
      axis_smoke_sequence::get_type());
    super.build_phase(phase);
  endfunction
endclass
```

15. Correct Handling of randc / Cyclic Stimulus

Do not use `randc: true` in the interface YAML unless your installed UVMF reference explicitly documents it. A portable approach is to implement cyclic selection in SystemVerilog.

True SystemVerilog randc helper

```
class axis_id_cycle;
  randc bit [3:0] id;
endclass

// Sequence use:
axis_id_cycle cyc = new();
repeat (16) begin
  assert(cyc.randomize());
  // constrain the UVMF transaction id == cyc.id
end
```

This guarantees each 4-bit value is selected once per `randc` cycle before repetition, subject to SystemVerilog `randc` semantics. If additional constraints make the cycle unsatisfiable, prefer an explicit shuffled queue so behavior remains deterministic and debuggable.

16. Functional Coverage and Assertions

Classic UVMF generates an interface transaction coverage class alongside the transaction and agent. Customize that generated covergroup, or add a dedicated analysis coverage component when packet-level context is required. Keep protocol assertions in SystemVerilog and bind/instantiate them at the signal level.

Coverage model (illustrative)

```
covergroup axis_cg with function sample(axis_out_transaction #( ) t,
                                     int unsigned stall_cycles,
                                     int unsigned packet_len);

  cp_keep: coverpoint t.keep {
    bins full    = {'1};
    bins partial[] = {[1:$]};
  }
  cp_last: coverpoint t.last;
  cp_id:   coverpoint t.id;
  cp_dest: coverpoint t.dest;
  cp_stall: coverpoint stall_cycles {
    bins none = {0};
  }
endgroup
```

```

    bins short[] = {[1:3]};
    bins long = {[4:32]};
  }
  cp_pkt_len: coverpoint packet_len {
    bins one = {1};
    bins short = {[2:4]};
    bins medium = {[5:16]};
    bins long = {[17:64]};
  }
  x_stall_pkt: cross cp_stall, cp_pkt_len;
endgroup

```

Protocol assertions

```

property p_hold_while_stalled;
  @(posedge aclk) disable iff (!areset_n)
    tvalid && !tready | => tvalid && $stable({tdata,tkeep,tlast,tid,tdest,tuser});
endproperty
assert property (p_hold_while_stalled);

property p_no_x_on_transfer;
  @(posedge aclk) disable iff (!areset_n)
    tvalid && tready |-> !$isunknown({tdata,tkeep,tlast,tid,tdest,tuser});
endproperty
assert property (p_no_x_on_transfer);

```

17. Test List and Regression Mapping

Test	Sequence	Primary constraints	Checks	Coverage
axis_smoke_test	axis_smoke_sequence	20 beats; full TKEEP; short gaps	Handshake, basic payload compare	handshake; full keep
axis_packet_len_test	axis_packet_length_sequence	packet_len 1..64; TLAST on final beat	TLAST/ordering	packet length x last
axis_partial_keep_test	axis_partial_keep_sequence	partial TKEEP only on final beat	Byte-lane preservation	partial keep bins
axis_sideband_test	axis_sideband_sequence	sweep ID/DEST/USER values	Sideband preservation	id/dest/user bins
axis_gap_stress_test	axis_gap_stress_sequence	gap 0..8; long traffic	No drop/duplicate	gap x packet len
axis_reset_midpacket_test	axis_reset_midpacket_sequence	assert reset during active packet	Reset flush/restart	reset scenarios
axis_backpressure_test	axis_backpressure_sequence	randomized output TREADY responder	Hold-under-stall; no loss	stall x packet len
axis_long_random_test	axis_long_random_sequence	thousands of constrained-random beats	Scoreboard + assertions	coverage closure

18. Project-Owned Regression Manifest

The following YAML is intentionally a project metadata file, not an asserted yaml2uvmf schema. A launcher script can translate it into simulator/VRM invocations. If you use Questa VRM, adapt this manifest to the exact testlist syntax documented by your installed release.

regression/axis_regression_plan.yaml

```

project: axis_pipe
bench: axis_pipe
runs:
  - test: axis_smoke_test
    seeds: 1
    tags: [smoke, quick]
  - test: axis_packet_len_test
    seeds: 10
    tags: [packet, nightly]
  - test: axis_partial_keep_test
    seeds: 10
    tags: [keep, nightly]
  - test: axis_sideband_test
    seeds: 10
    tags: [sideband, nightly]
  - test: axis_reset_midpacket_test
    seeds: 20
    tags: [reset, corner]
  - test: axis_backpressure_test
    seeds: 50
    tags: [stall, stress]
  - test: axis_long_random_test
    seeds: 100
    tags: [random, coverage, full]

coverage_goals:
  functional: 100
  assertions: 100
  code_statement: 95
  code_branch: 90

```

19. Requirements Traceability Matrix

Requirement	Tests	Checker	Coverage	Exit criterion
AXIS-REQ-001	smoke, all random	Handshake monitor	cp_handshake	Every transfer seen only on VALID&&READY
AXIS-REQ-002	backpressure	p_hold_while_stalled	cp_stall	No assertion failures
AXIS-REQ-003	packet_len	TLAST sequence/monitor	cp_pkt_len x cp_last	TLAST only at expected packet end
AXIS-REQ-004	partial_keep	scoreboard	cp_keep	Full + partial bins hit
AXIS-REQ-005	reset_midpacket	reset assertions	cp_reset	No stale transfer after reset
AXIS-REQ-006	all	in-order scoreboard	transaction count	Expected == actual, no residual entries
AXIS-REQ-007	sideband	scoreboard	cp_id/cp_dest/cp_user	All intended sideband bins hit
AXIS-REQ-008	backpressure, stress	scoreboard + counters	stall x packet len	Accepted input count == output count

20. End-to-End Build and Verification Workflow

1. Freeze the AXI4-Stream subset: widths, optional sidebands, packet/TKEEP policy, reset behavior, and expected DUT transform.
2. Encode reusable pin/transaction structure in axis_in_interface.yaml and axis_out_interface.yaml.
3. Generate interfaces and inspect the transaction classes, signal bundle interfaces, driver/monitor BFM and custom regions.
4. Define axis_predictor.yaml and axis_environment.yaml; verify TLM endpoint names before generation.
5. Define axis_bench.yaml with top_env, BFM activity, clock and reset settings.

6. Run `yaml2uvmf.py` with the complete YAML set.
7. Complete generated BFM protocol tasks and predictor custom code.
8. Edit `hdl_top.sv` custom region to instantiate and wire the DUT; add RTL/file-list entries.
9. Add custom sequences, tests, coverage and assertions using generated base classes/types.
10. Run smoke first, then directed corners, then constrained-random/stress. Close scoreboard errors and assertion failures before coverage chasing.
11. Run the full regression across multiple seeds; merge coverage using your simulator/VRM flow.
12. Review the traceability matrix and sign off only when every requirement has passing checks and its intended coverage goal is met.

21. Debug Checklist

Symptom	Likely cause	Action
No input activity	<code>axis_in_agent</code> is PASSIVE or top sequence not started	Check bench <code>active_passive</code> and test/sequence selection.
Driver changes payload under stall	BFM waits one cycle but does not loop until TREADY	Hold TVALID and payload stable until handshake.
Scoreboard mismatches all beats	Predictor and output transaction field mapping differ	Compare field names/types and predictor assignment.
Scoreboard missing output	<code>M_AXIS TREADY</code> not asserted or output monitor not handshake-gated	Check <code>hdl_top</code> tie/responder and monitor condition.
Generator rejects YAML key	Key belongs to another schema/release or is pseudo-YAML	Compare with installed UVMF YAML Reference; remove unsupported key.
<code>randc</code> repeats unexpectedly	<code>randc</code> emulation not actually excluding history	Use native SV <code>randc</code> helper or shuffled queue.
Coverage is 0	Coverage component not constructed/subscribed or sampling only non-handshake cycles	Connect <code>monitored_ap</code> and sample only accepted transfers.
Reset causes false mismatches	Scoreboard/predictor state not flushed consistently	Implement reset-aware flush policy and document expected DUT behavior.

22. YAML Quick Reference — Safe Core Keys

Scope	Core keys used in this manual
Interface	<code>clock</code> , <code>reset</code> , <code>reset_assertion_level</code> , <code>veloce_ready</code> , <code>parameters</code> , <code>ports</code> , <code>transaction_vars</code> , <code>transaction_constraints</code> , <code>config_vars</code>
Utility component	<code>util_components</code> , <code>type: predictor</code> , <code>analysis_exports</code> , <code>analysis_ports</code>
Environment	<code>agents</code> , <code>initiator_responder</code> , <code>analysis_components</code> , <code>scoreboards</code> , <code>tlm_connections</code> , <code>subenvs</code> (when used)
Bench	<code>top_env</code> , <code>active_passive</code> , <code>clock_half_period</code> , <code>clock_phase_offset</code> , <code>reset_assertion_level</code> , <code>reset_duration</code> , <code>interface_params</code>
Not treated as universal <code>yaml2uvmf</code> keys	<code>protocol</code> , <code>dut</code> , <code>map</code> , <code>sequences</code> , <code>tests</code> , <code>regression</code> , <code>coverage</code> , <code>randc</code>

23. Migration Notes from the Previous Manual

Previous pattern	Updated handling
transaction: fields: rand: true	transaction_vars with isrand / iscompare
reset_polarity: active_low	reset_assertion_level: "False" for active-low reset
randc: true in interface YAML	Implement cyclic behavior in custom SystemVerilog
coverage: covergroups: directly under interface YAML	Customize the generated transaction coverage class or a dedicated SV coverage component; do not assume a portable coverage YAML key
bench dut: module/file/ports mapping	Instantiate/wire DUT in generated hdl_top.sv custom region
sequences YAML	Use generated environment sequence base and custom SV sequences
tests/regression YAML passed to yaml2uvmf	Keep regression/testlist orchestration separate unless your installed tool documents that schema
ALU + AXI4-Stream mixed example	Single AXI4-Stream pipeline project from requirement through regression

24. Completion Checklist

- ☐ All five generator YAML files parse successfully with the installed yaml2uvmf.py.
- ☐ Generated interface packages compile without manual edits outside UVMF custom regions.
- ☐ DUT is connected in hdl_top.sv and all RTL/file-list dependencies are present.
- ☐ Input driver obeys AXI4-Stream VALID/READY persistence rules.
- ☐ Input and output monitors emit exactly one transaction per accepted beat.
- ☐ Predictor reflects DUT functionality and scoreboard drains to zero outstanding items.
- ☐ Protocol assertions pass across reset, idle, throughput and stall scenarios.
- ☐ Directed tests cover TLAST, TKEEP, sideband and reset corner cases.
- ☐ Random/stress tests execute across multiple seeds.
- ☐ Traceability matrix shows a passing checker and coverage evidence for every requirement.

25. Reference Basis

Primary schema authority for actual use: the UVMF documentation installed with your tool release under \$UVMF_HOME/docs, especially UVMF_Code_Generator_YAML_Reference.pdf and UVM_Framework_Users_Guide.pdf.

- UVMF 2019.4 ALU tutorial examples confirm transaction_vars / transaction_constraints, environment agents + TLM connections, and bench keys such as top_env, active_passive, clock_half_period and reset_duration.
- Current public HDL Verifier documentation continues to show yaml2uvmf.py as the UVMF YAML-to-testbench generation entry point and generated project_benches / verification_ip structure.
- AXI4-Stream protocol semantics used here follow the standard VALID/READY transfer model; adapt optional sideband widths and packet policy to the DUT specification.